

Escuela Politécnica Nacional

Métodos Numericos

Erick Cuichan

Tarea 9

link repositorio: [Tarea09](#)

Ejercicio 1

Para cada uno de los siguientes sistemas lineales, obtenga, de ser posible, una solución con métodos gráficos. Explique los resultados desde un punto de vista geométrico.

a)

$$x_1 + 2x_2 = 0$$

$$x_1 - x_2 = 0$$

De la segunda ecuación:

$$x_1 = x_2$$

Sustituyendo en la primera:

$$x_2 + 2x_2 = 0$$

$$3x_2 = 0$$

$$x_2 = 0$$

Entonces:

$$x_1 = 0$$

$$(x_1, x_2) = (0, 0)$$

Interpretación geométrica: las dos rectas se cortan en un solo punto, por lo tanto el sistema tiene solución única.

```
In [28]: import numpy as np
import matplotlib.pyplot as plt

x1 = np.linspace(-10, 10, 400)

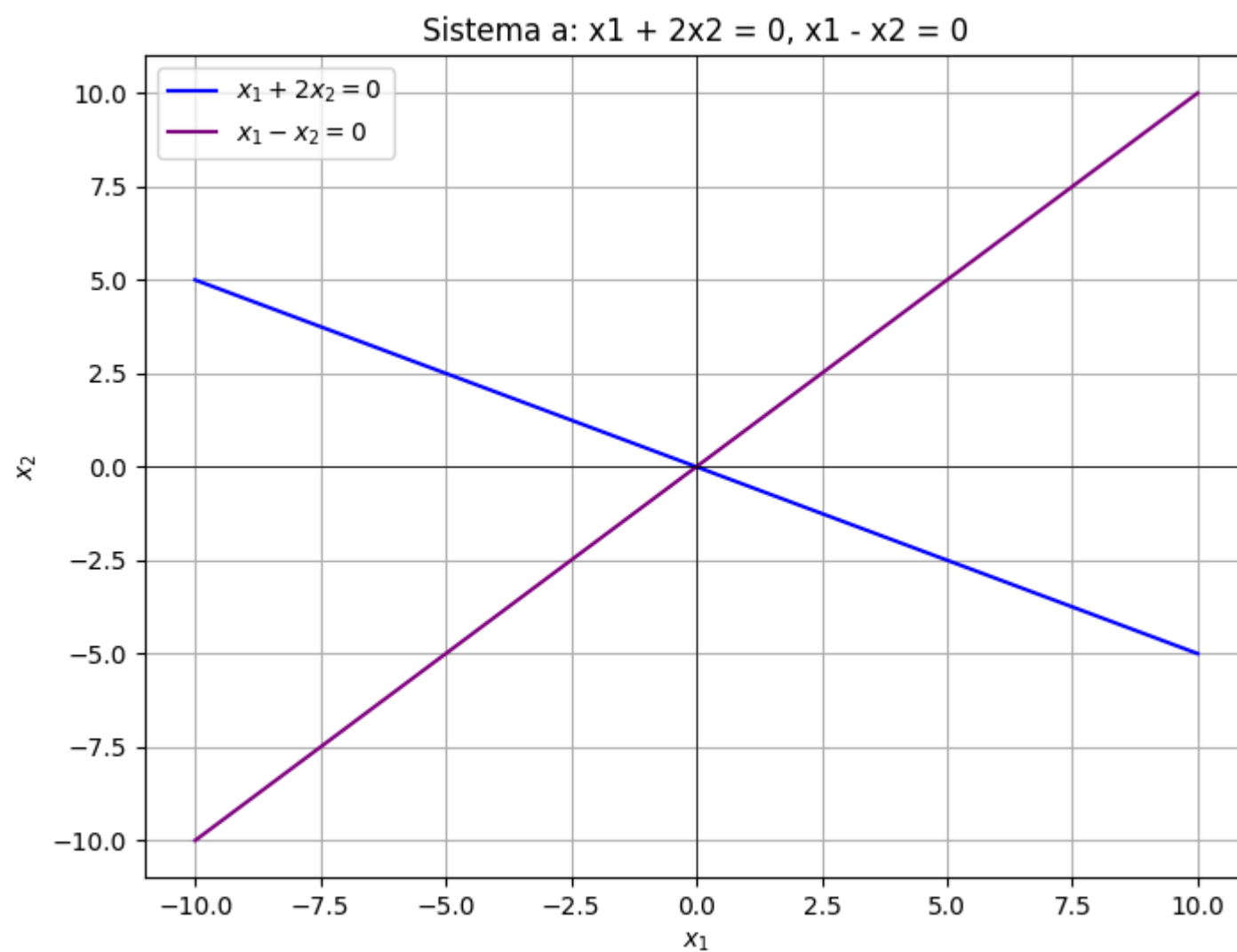
x2_1 = -x1/2

x2_2 = x1

plt.figure(figsize=(8, 6))
plt.plot(x1, x2_1, label=r'$x_1 + 2x_2 = 0$', color='blue')
plt.plot(x1, x2_2, label=r'$x_1 - x_2 = 0$', color='purple')

plt.xlabel(r'$x_1$')
plt.ylabel(r'$x_2$')
plt.title('Sistema a: $x_1 + 2x_2 = 0$, $x_1 - x_2 = 0$')
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(True)
plt.legend()

plt.show()
```



b)

$$\begin{aligned}x_1 + 2x_2 &= 3 \\ -2x_1 - 4x_2 &= 6\end{aligned}$$

Dividiendo la segunda ecuación para (-2):

$$x_1 + 2x_2 = -3$$

Entonces se tiene:

$$\begin{aligned}x_1 + 2x_2 &= 3 \\ x_1 + 2x_2 &= -3\end{aligned}$$

Las rectas tienen la misma pendiente, pero diferente término independiente.

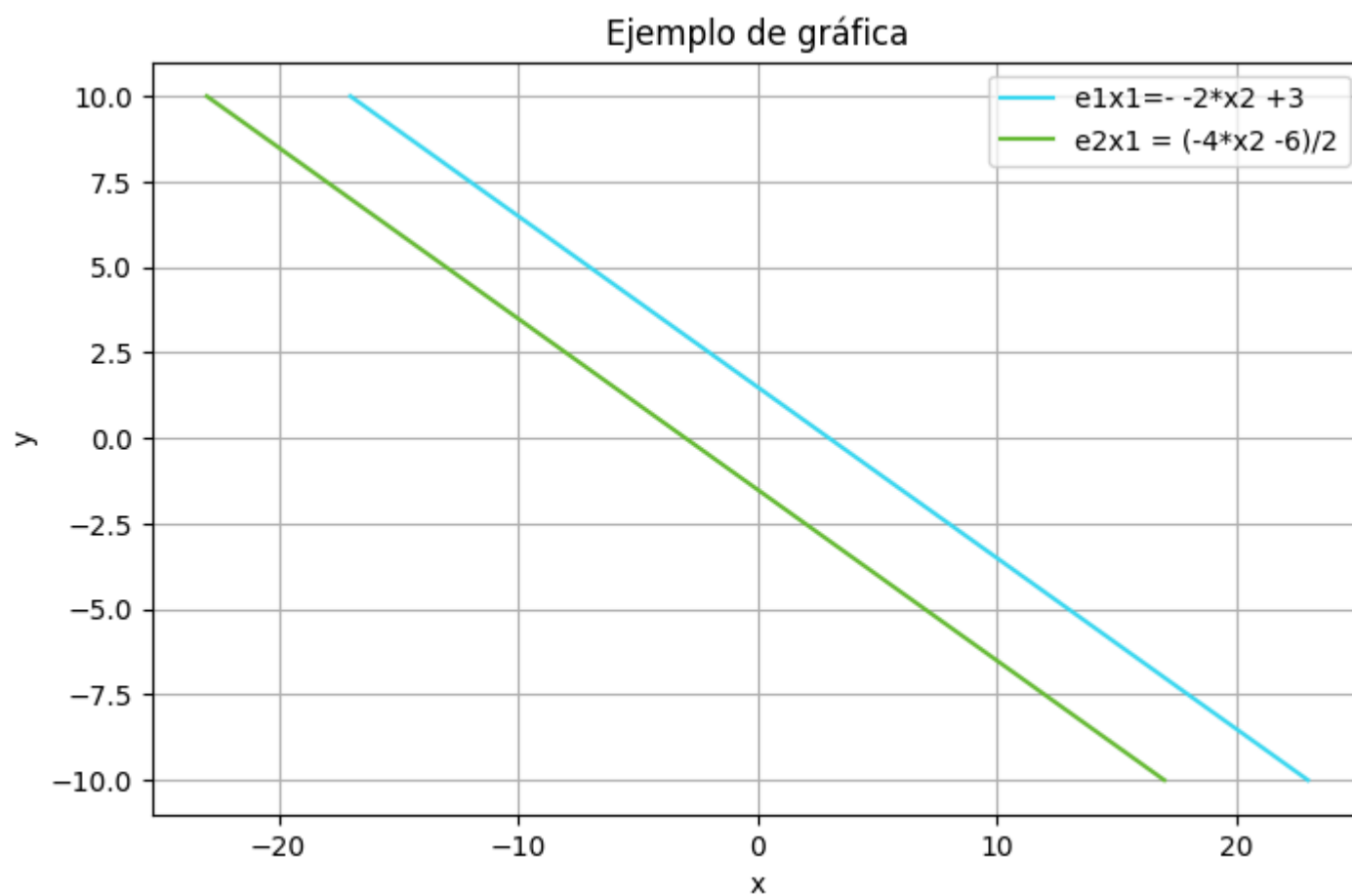
Respuesta:

No tiene solución

Interpretación geométrica: las rectas son paralelas y no se intersectan.

```
In [29]: x2 = np.linspace(-10, 10, 100)
x1e1= 3 - 2* x2
x1e2= (-4*x2 -6)/2

plt.figure(figsize=(8, 5))
plt.plot(x1e1, x2,color="#3BDAF3", label='e1x1=- 2*x2 +3')
plt.plot(x1e2, x2, color="#64BB32", label='e2x1 = (-4*x2 -6)/2')
plt.title('Ejemplo de gráfica')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()
```



c)

$$2x_1 + x_2 = -1$$

$$x_1 + x_2 = 2$$

$$x_1 - 3x_2 = 5$$

Restando la segunda ecuación de la primera:

$$(2x_1 + x_2) - (x_1 + x_2) = -1 - 2$$

$$x_1 = -3$$

Sustituyendo en:

$$x_1 + x_2 = 2$$

$$-3 + x_2 = 2$$

$$x_2 = 5$$

Verificando en la tercera ecuación:

$$x_1 - 3x_2 = 5$$

$$-3 - 3(5) = 5$$

$$-18 \neq 5$$

Respuesta:

No tiene solución

Interpretación geométrica: las tres rectas no se intersectan en un mismo punto.

```
In [30]: x2_1_c = -2*x1 - 1

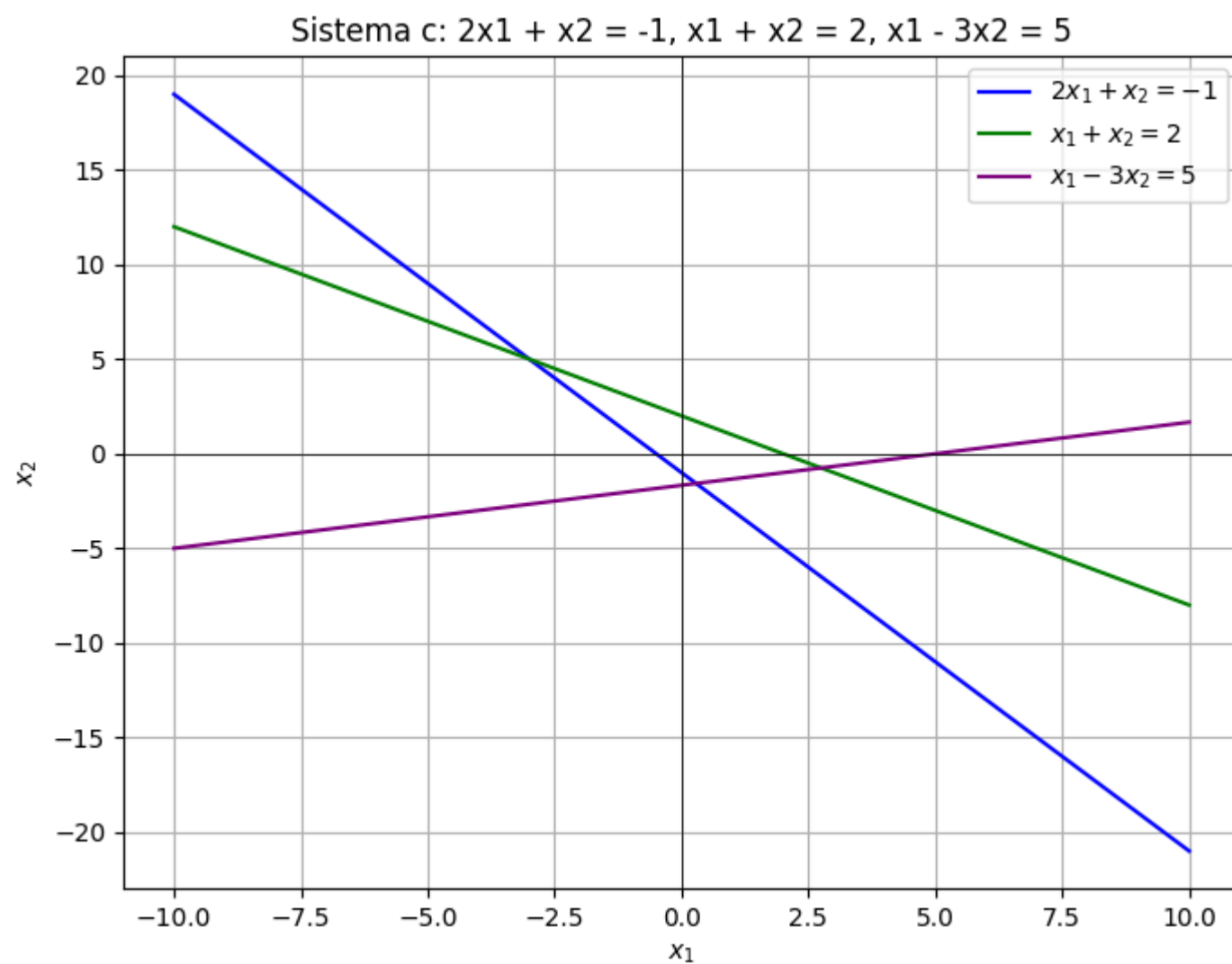
x2_2_c = 2 - x1

x2_3_c = (x1 - 5) / 3

plt.figure(figsize=(8, 6))
plt.plot(x1, x2_1_c, label=r'$2x_1 + x_2 = -1$', color='blue')
plt.plot(x1, x2_2_c, label=r'$x_1 + x_2 = 2$', color='green')
plt.plot(x1, x2_3_c, label=r'$x_1 - 3x_2 = 5$', color='purple')

plt.xlabel(r'$x_1$')
plt.ylabel(r'$x_2$')
plt.title('Sistema c: 2x1 + x2 = -1, x1 + x2 = 2, x1 - 3x2 = 5')
plt.axhline(0, color='black',linewidth=0.5)
plt.axvline(0, color='black',linewidth=0.5)
plt.grid(True)
plt.legend()

plt.show()
```



d)

$$2x_1 + x_2 + x_3 = 1$$

$$2x_1 + 4x_2 - x_3 = -1$$

Sea:

$$x_3 = t$$

Entonces:

$$2x_1 + x_2 = 1 - t$$

$$2x_1 + 4x_2 = -1 + t$$

Restando:

$$3x_2 = -2 + 2t$$

$$x_2 = \frac{-2 + 2t}{3}$$

Ahora:

$$2x_1 = 1 - t - x_2$$

$$2x_1 = 1 - t - \frac{-2 + 2t}{3}$$

$$2x_1 = \frac{5 - 5t}{3}$$

$$x_1 = \frac{5 - 5t}{6}$$

Respuesta:

$$x_1 = \frac{5 - 5t}{6}, \quad x_2 = \frac{-2 + 2t}{3}, \quad x_3 = t$$

Interpretación geométrica: son dos planos en el espacio que se intersectan en una recta. Por lo tanto, existen infinitas soluciones.

In [31]: `from mpl_toolkits.mplot3d import Axes3D`

```
x1_4 = np.linspace(-10, 10, 200)
x2_4 = np.linspace(-10, 10, 200)
X1, X2 = np.meshgrid(x1_4, x2_4)

x3_1_d = 1 - 2*X1 - X2
x3_2_d = 2*X1 + 4*X2 + 1
```

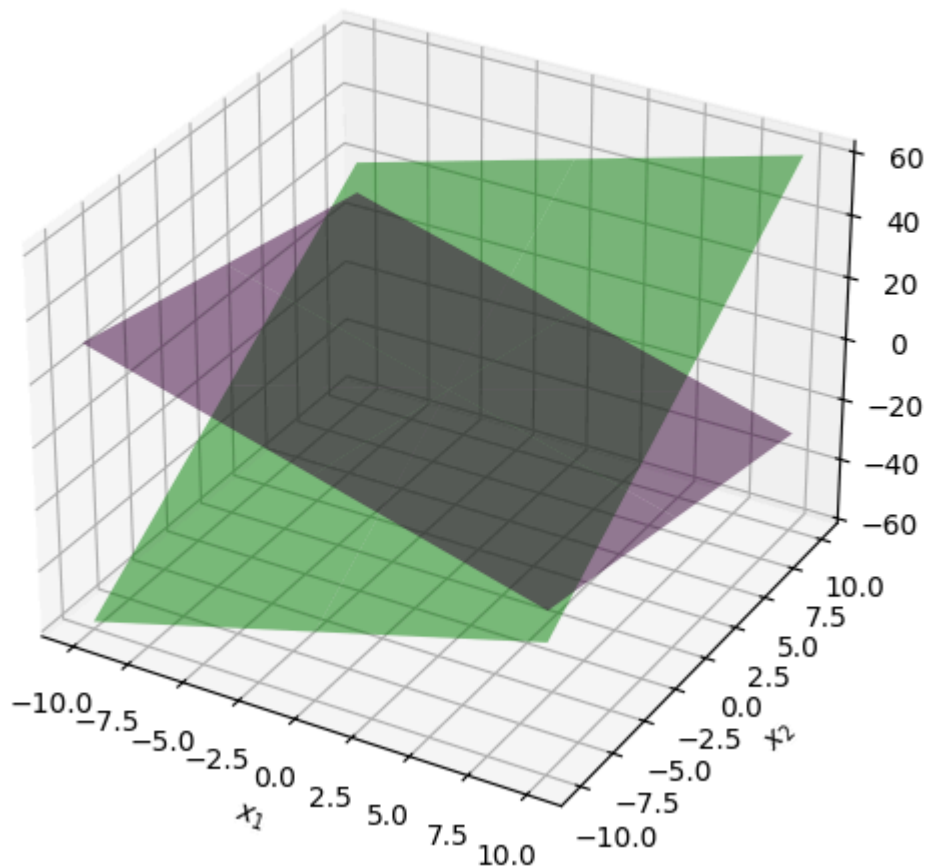
```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X1, X2, x3_1_d, color='Purple', alpha=0.5, rstride=100, cstride=100)
ax.plot_surface(X1, X2, x3_2_d, color='green', alpha=0.5, rstride=100, cstride=100)

ax.set_xlabel(r'$x_1$')
ax.set_ylabel(r'$x_2$')
ax.set_zlabel(r'$x_3$')
ax.set_title('Sistema d:  $2x_1 + x_2 + x_3 = 1$ ,  $2x_1 + 4x_2 - x_3 = -1$ ')

plt.show()
```

Sistema d: $2x_1 + x_2 + x_3 = 1$, $2x_1 + 4x_2 - x_3 = -1$



Ejercicio 2

Utilice la eliminación gaussiana con sustitución hacia atrás y aritmética de redondeo de dos dígitos para resolver los siguientes sistemas lineales.

No reordene las ecuaciones.

La solución exacta para cada sistema es:

$$x_1 = -1, \quad x_2 = 2, \quad x_3 = 3$$

a)

$$-x_1 + 4x_2 + x_3 = 8$$

$$\frac{5}{3}x_1 + \frac{2}{3}x_2 + \frac{2}{3}x_3 = 1$$

$$2x_1 + x_2 + 4x_3 = 11$$

```
In [32]: def gauss_elimination(A, b):
n = len(b)
# Elimination phase
for i in range(n):
    # Pivoting: swap rows if necessary
    if A[i,i] == 0:
        for j in range(i+1, n):
            if A[j,i] != 0:
                A[[i,j]] = A[[j,i]]
                b[i], b[j] = b[j], b[i]
                break
    for j in range(i+1, n):
        if A[j,i] != 0:
            factor = A[j,i] / A[i,i]
            A[j] = A[j] - factor * A[i]
            b[j] = b[j] - factor * b[i]

# Back substitution phase
x = np.zeros(n)
```

```
for i in range(n-1, -1, -1):
    x[i] = (b[i] - np.dot(A[i,i+1:], x[i+1:])) / A[i,i]

return x

A_a = np.array([[ -1, 4, 1], [5/3, 2/3, 2/3], [2, 1, 4]], dtype=float)
b_a = np.array([8, 1, 11], dtype=float)

x_a = gauss_elimination(A_a, b_a)

print("Solución del sistema a:", np.round(x_a, 2))
```

Solución del sistema a: [-1. 1. 3.]

b)

$$4x_1 + 2x_2 - x_3 = -5$$
$$\frac{1}{9}x_1 + \frac{1}{9}x_2 - \frac{1}{3}x_3 = -1$$
$$x_1 + 4x_2 + 2x_3 = 9$$

```
In [33]: A_b = np.array([[4, 2, -1], [1/9, 1/9, -1/3], [1, 4, 2]], dtype=float)
b_b = np.array([-5, -1, 9], dtype=float)

x_b = gauss_elimination(A_b, b_b)
print("Solución del sistema b:", np.round(x_b, 2))
```

Solución del sistema b: [-1. 1. 3.]

Ejercicio 3

Utilice el algoritmo de eliminación gaussiana para resolver, de ser posible, los siguientes sistemas lineales, y determine si se necesitan intercambios de fila.

a)

$$x_1 - x_2 + 3x_3 = 2$$
$$3x_1 - 3x_2 + x_3 = -1$$
$$x_1 + x_2 = 3$$

```
In [34]: import logging
from sys import stdout
from datetime import datetime

intercambio=0
logging.basicConfig(
    level=logging.INFO,
    format="[%asctime)s][%(levelname)s] %(message)s",
    stream=stdout,
    datefmt="%m-%d %H:%M:%S",
)
logging.info(datetime.now())

def eliminacion_gaussiana(A: np.ndarray | list[list[float | int]]) -> np.ndarray:

    if not isinstance(A, np.ndarray):
        logging.debug("Convirtiendo A a numpy array.")
        A = np.array(A)
    assert A.shape[0] == A.shape[1] - 1, "La matriz A debe ser de tamaño n-by-(n+1)."
    n = A.shape[0]

    for i in range(0, n - 1): # Loop por columna

        # --- encontrar pivote
        p = None # default, first element
        for pi in range(i, n):
            if A[pi, i] != 0:
                # must be nonzero
                continue

            if p is None:
                # first nonzero element
                p = pi
                continue

            if abs(A[pi, i]) < abs(A[p, i]):
                p = pi

        if p is None:
```



```

        # no pivot found.
        raise ValueError("No existe solución única.")

    if p != i:
        # swap rows
        logging.debug(f"Intercambiando filas {i} y {p}")
        _aux = A[i, :].copy()
        A[i, :] = A[p, :].copy()
        A[p, :] = _aux
        global intercambio
        intercambio+=1

    # --- Eliminación: loop por fila
    for j in range(i + 1, n):
        m = A[j, i] / A[i, i]
        A[j, i:] = A[j, i:] - m * A[i, i:]

    logging.info(f"\n{A}")

    if A[n - 1, n - 1] == 0:
        raise ValueError("No existe solución única.")

    print(f"\n{A}")
    # --- Sustitución hacia atrás
    solucion = np.zeros(n)
    solucion[n - 1] = A[n - 1, n] / A[n - 1, n - 1]

    for i in range(n - 2, -1, -1):
        suma = 0
        for j in range(i + 1, n):
            suma += A[i, j] * solucion[j]
        solucion[i] = (A[i, n] - suma) / A[i, i]

    print("Número de intercambios de filas:", intercambio)
    return solucion

```

[07-08 13:59:58][INFO] 2026-07-08 13:59:58.579535

```

In [35]: A_a = [
          [1., -1, 3, 2],
          [3, -3, 1, -1],
          [1, 1, 0, 3]
        ]

        eliminacion_gaussiana(A_a)

```

[07-08 13:59:58][INFO]

```

[[ 1. -1.  3.  2.]
 [ 0.  0. -8. -7.]
 [ 0.  2. -3.  1.]]

```

[07-08 13:59:58][INFO]

```

[[ 1. -1.  3.  2.]
 [ 0.  2. -3.  1.]
 [ 0.  0. -8. -7.]]

```

Número de intercambios de filas: 1

Out[35]: array([1.1875, 1.8125, 0.875])

b)

$$2x_1 - 1.5x_2 + 3x_3 = 1$$

$$-x_1 + 2x_3 = 3$$

$$4x_1 - 4.5x_2 + 5x_3 = 1$$

```

In [36]: A_b = [[2., -1/5, 3, 1], [-1, 0, 2, 3], [4, -4.5, 5, 1]]

        eliminacion_gaussiana(A_b)

```

[07-08 13:59:58][INFO]

```

[[-1.  0.  2.  3.]
 [ 0. -0.2  7.  7.]
 [ 0. -4.5 13. 13.]]

```

[07-08 13:59:58][INFO]

```

[[ -1.  0.  2.  3.]
 [  0. -0.2  7.  7.]
 [  0.  0. -144.5 -144.5]]

```

Número de intercambios de filas: 2

Out[36]: array([-1., -0., 1.])

c)

$$2x_1 = 3$$

$$x_1 + 1.5x_2 = 4.5$$

$$-3x_2 + 0.5x_3 = -6.6$$

$$2x_1 - 2x_2 + x_3 + x_4 = 0.8$$

```
In [37]: A_c = [
    [2.,0,0,0,3],
    [1,1.5,0,0,4.5],
    [0,-3,0.5,0,-6.6],
    [2,-2,1,1,0.8]
]
eliminacion_gaussiana(A_c)

[07-08 13:59:58][INFO]
[[ 1.  1.5  0.  0.  4.5]
 [ 0. -3.  0.  0. -6. ]
 [ 0. -3.  0.5  0. -6.6]
 [ 0. -5.  1.  1. -8.2]]
[07-08 13:59:58][INFO]
[[ 1.  1.5  0.  0.  4.5]
 [ 0. -3.  0.  0. -6. ]
 [ 0.  0.  0.5  0. -0.6]
 [ 0.  0.  1.  1.  1.8]]
[07-08 13:59:58][INFO]
[[ 1.  1.5  0.  0.  4.5]
 [ 0. -3.  0.  0. -6. ]
 [ 0.  0.  0.5  0. -0.6]
 [ 0.  0.  0.  1.  3. ]]
Número de intercambios de filas: 3
Out[37]: array([ 1.5,  2. , -1.2,  3. ])
```

d)

$$x_1 + x_2 + x_4 = 2$$

$$2x_1 + x_2 - x_3 + x_4 = 1$$

$$4x_1 - x_2 - 2x_3 + 2x_4 = 0$$

$$3x_1 - x_2 - x_3 + 2x_4 = -3$$

```
In [38]: A_d = [
    [1,1,0,1,2],
    [2,1,-1,1,1],
    [4,-1,-2,2,0],
    [3,-1,-1,2,-3]
]
eliminacion_gaussiana(A_d)

[07-08 13:59:58][INFO]
[[ 1  1  0  1  2]
 [ 0 -1 -1 -1 -3]
 [ 0 -5 -2 -2 -8]
 [ 0 -4 -1 -1 -9]]
[07-08 13:59:58][INFO]
[[ 1  1  0  1  2]
 [ 0 -1 -1 -1 -3]
 [ 0  0  3  3  7]
 [ 0  0  3  3  3]]
[07-08 13:59:58][INFO]
[[ 1  1  0  1  2]
 [ 0 -1 -1 -1 -3]
 [ 0  0  3  3  7]
 [ 0  0  0  0 -4]]

-----
ValueError                                Traceback (most recent call last)
Cell In[38], line 8
      1 A_d = [
      2     [1,1,0,1,2],
      3     [2,1,-1,1,1],
      4     [4,-1,-2,2,0],
      5     [3,-1,-1,2,-3]
      6 ]
----> 8 eliminacion_gaussiana(A_d)

Cell In[34], line 60, in eliminacion_gaussiana(A)
     57 logging.info(f"\n{A}")
     59 if A[n - 1, n - 1] == 0:
--> 60     raise ValueError("No existe solución única.")
     62     print(f"\n{A}")
     63 # --- Sustitución hacia atrás

ValueError: No existe solución única.
```


Ejercicio 4

Use el algoritmo de eliminación gaussiana y la aritmética computacional de precisión de 32 bits para resolver los siguientes sistemas lineales.

a)

$$\frac{1}{4}x_1 + \frac{1}{5}x_2 + \frac{1}{6}x_3 = 9$$
$$\frac{1}{3}x_1 + \frac{1}{4}x_2 + \frac{1}{5}x_3 = 8$$
$$\frac{1}{2}x_1 + x_2 + 2x_3 = 8$$

```
In [ ]: intercambio=0
logging.basicConfig(
    level=logging.INFO,
    format="[%(asctime)s][%(levelname)s] %(message)s",
    stream=stdout,
    datefmt="%m-%d %H:%M:%S",
)
logging.info(datetime.now())

def eliminacion_gaussiana(A: np.ndarray | list[list[float | int]]) -> np.ndarray:
    if not isinstance(A, np.ndarray):
        logging.debug("Convirtiendo A a numpy array.")
        A = np.array(A, dtype=np.float32) # Se aplica la precision de 32 bits
    assert A.shape[0] == A.shape[1] - 1, "La matriz A debe ser de tamaño n-by-(n+1)."
```

```
    n = A.shape[0]

    for i in range(0, n - 1):

        p = None
        for pi in range(i, n):
            if A[pi, i] != 0:
                continue

            if p is None:
                p = pi
                continue

            if abs(A[pi, i]) < abs(A[p, i]):
                p = pi

        if p is None:
            raise ValueError("No existe solución única.")

        if p != i:
            logging.debug(f"Intercambiando filas {i} y {p}")
            _aux = A[i, :].copy()
            A[i, :] = A[p, :].copy()
            A[p, :] = _aux
            global intercambio
            intercambio+=1

        for j in range(i + 1, n):
            m = A[j, i] / A[i, i]
            A[j, i:] = A[j, i:] - m * A[i, i:]

        logging.info(f"\n{A}")

    if A[n - 1, n - 1] == 0:
        raise ValueError("No existe solución única.")

    print(f"\n{A}")
    solucion = np.zeros(n)
    solucion[n - 1] = A[n - 1, n] / A[n - 1, n - 1]

    for i in range(n - 2, -1, -1):
        suma = 0
        for j in range(i + 1, n):
            suma += A[i, j] * solucion[j]
        solucion[i] = (A[i, n] - suma) / A[i, i]

    print("Número de intercambios de filas:", intercambio)
    return solucion
```

[07-08 13:59:02][INFO] 2026-07-08 13:59:02.480745

```
In [39]: A_aa = [
    [1/4,1/5,1/6,9],
    [1/3,1/4,1/5,8],
```

```
[1/2,1,2,8]
]

eliminacion_gaussiana(A_aa)
```

```
[07-08 14:00:08][INFO]
[[ 0.25      0.2      0.16666667   9.      ]
 [ 0.      -0.01666667 -0.02222222  -4.      ]
 [ 0.      0.6      1.66666667 -10.     ]]
[07-08 14:00:08][INFO]
[[ 2.50000000e-01  2.00000000e-01  1.66666667e-01  9.00000000e+00]
 [ 0.00000000e+00 -1.66666667e-02 -2.22222222e-02 -4.00000000e+00]
 [ 0.00000000e+00  0.00000000e+00  8.66666667e-01 -1.54000000e+02]]
Número de intercambios de filas: 3
```

Out[39]: array([-227.07692308, 476.92307692, -177.69230769])

b)

$$3.333x_1 + 15920x_2 - 10.333x_3 = 15913$$

$$2.222x_1 + 16.71x_2 + 9.612x_3 = 28.544$$

$$1.5611x_1 + 5.1791x_2 + 1.6852x_3 = 8.4254$$

```
In [40]: A_ab = [
          [3.333,15920,-10.333,15913],
          [2.222,16.71,9.612,28.544],
          [1.5611,5.1791,1.6852,8.4254]
        ]

eliminacion_gaussiana(A_ab)
```

```
[07-08 14:00:10][INFO]
[[ 1.56110000e+00  5.17910000e+00  1.68520000e+00  8.42540000e+00]
 [ 0.00000000e+00  9.33830043e+00  7.21336160e+00  1.65516620e+01]
 [ 0.00000000e+00  1.59089425e+04 -1.39309576e+01  1.58950115e+04]]
[07-08 14:00:10][INFO]
[[ 1.56110000e+00  5.17910000e+00  1.68520000e+00  8.42540000e+00]
 [ 0.00000000e+00  9.33830043e+00  7.21336160e+00  1.65516620e+01]
 [ 0.00000000e+00  0.00000000e+00 -1.23027790e+04 -1.23027790e+04]]
Número de intercambios de filas: 4
```

Out[40]: array([1., 1., 1.])

c)

$$x_1 + \frac{1}{2}x_2 + \frac{1}{3}x_3 + \frac{1}{4}x_4 = \frac{1}{6}$$

$$\frac{1}{2}x_1 + \frac{1}{3}x_2 + \frac{1}{4}x_3 + \frac{1}{5}x_4 = \frac{1}{7}$$

$$\frac{1}{3}x_1 + \frac{1}{4}x_2 + \frac{1}{5}x_3 + \frac{1}{6}x_4 = \frac{1}{8}$$

$$\frac{1}{4}x_1 + \frac{1}{5}x_2 + \frac{1}{6}x_3 + \frac{1}{7}x_4 = \frac{1}{9}$$

```
In [41]: A_ac = [
          [1,1/2,1/3,1/4,1/6],
          [1/2,1/3,1/4,1/5,1/7],
          [1/3,1/4,1/5,1/6,1/8],
          [1/4,1/5,1/6,1/7,1/9]
        ]

eliminacion_gaussiana(A_ac)
```

```
[07-08 14:00:12][INFO]
[[ 0.25      0.2      0.16666667  0.14285714  0.11111111]
 [ 0.      -0.06666667 -0.08333333 -0.08571429 -0.07936508]
 [ 0.      -0.01666667 -0.02222222 -0.02380952 -0.02314815]
 [ 0.      -0.3      -0.33333333 -0.32142857 -0.27777778]]
[07-08 14:00:12][INFO]
[[ 0.25      0.2      0.16666667  0.14285714  0.11111111]
 [ 0.      -0.01666667 -0.02222222 -0.02380952 -0.02314815]
 [ 0.      0.      0.00555556  0.00952381  0.01322751]
 [ 0.      0.      0.06666667  0.10714286  0.13888889]]
[07-08 14:00:12][INFO]
[[ 0.25      0.2      0.16666667  0.14285714  0.11111111]
 [ 0.      -0.01666667 -0.02222222 -0.02380952 -0.02314815]
 [ 0.      0.      0.00555556  0.00952381  0.01322751]
 [ 0.      0.      0.      -0.00714286 -0.01984127]]
Número de intercambios de filas: 6
```

Out[41]: array([-0.03174603, 0.5952381 , -2.38095238, 2.77777778])

d)

$$\begin{aligned}2x_1 + x_2 - x_3 + x_4 - 3x_5 &= 7 \\ x_1 + 2x_3 - x_4 + x_5 &= 2 \\ -2x_2 - x_3 + x_4 - x_5 &= -5 \\ 3x_1 + x_2 - 4x_3 + 5x_5 &= 6 \\ x_1 - x_2 - x_3 - x_4 + x_5 &= -3\end{aligned}$$

```
In [43]: A_ad = [
    [2.,1,-1,1,-3,7],
    [1,0,2,-1,1,2],
    [0,-2,-1,1,-1,-5],
    [3,1,-4,0,5,6],
    [1,-1,-1,-1,1,-3]
]

eliminacion_gaussiana(A_ad)

[07-08 14:02:28][INFO]
[[ 1.  0.  2. -1.  1.  2.]
 [ 0.  1. -5.  3. -5.  3.]
 [ 0. -2. -1.  1. -1. -5.]
 [ 0.  1. -10.  3.  2.  0.]
 [ 0. -1. -3.  0.  0. -5.]]
[07-08 14:02:28][INFO]
[[ 1.  0.  2. -1.  1.  2.]
 [ 0.  1. -5.  3. -5.  3.]
 [ 0.  0. -11.  7. -11.  1.]
 [ 0.  0. -5.  0.  7. -3.]
 [ 0.  0. -8.  3. -5. -2.]]
[07-08 14:02:28][INFO]
[[ 1.  0.  2. -1.  1.  2. ]
 [ 0.  1. -5.  3. -5.  3. ]
 [ 0.  0. -5.  0.  7. -3. ]
 [ 0.  0.  0.  7. -26.4  7.6]
 [ 0.  0.  0.  3. -16.2  2.8]]
[07-08 14:02:28][INFO]
[[ 1.          0.          2.          -1.          1.
  2.          ]
 [ 0.          1.          -5.          3.          -5.
  3.          ]
 [ 0.          0.          -5.          0.          7.
 -3.          ]
 [ 0.          0.          0.          7.         -26.4
  7.6         ]
 [ 0.          0.          0.          3.         -16.2
  2.8         ]
 [ 0.          0.          0.          0.          11.4
 1.06666667]]
Número de intercambios de filas: 12
Out[43]: array([1.88304094, 2.80701754, 0.73099415, 1.43859649, 0.09356725])
```

Ejercicio 5

Dado el sistema lineal:

$$\begin{aligned}x_1 - x_2 + \alpha x_3 &= -2 \\ -x_1 + 2x_2 - \alpha x_3 &= 3 \\ \alpha x_1 + x_2 + x_3 &= 2\end{aligned}$$

Realice lo siguiente:

a)

Encuentre el valor o los valores de (α) para los que el sistema no tiene soluciones.

b)

Encuentre el valor o los valores de (α) para los que el sistema tiene un número infinito de soluciones.

c)

Suponga que existe una única solución para una (α) determinada, encuentre la solución.

In []: `import sympy`

```
def analyze_system_with_parameter(A_symbolic, b_symbolic, symbol):

    if not A_symbolic.is_square:
        print("Error: La matriz de coeficientes debe ser cuadrada.")
        return

    print("--- Análisis del Determinante ---")

    det_A = A_symbolic.det()
    print(f"La matriz de coeficientes A es:\n{A_symbolic}")
    print(f"\nEl determinante de A es: det(A) = {sympy.simplify(det_A)}")

    print("\nSe busca para qué valores del símbolo el determinante es cero.")
    critical_values = sympy.solve(det_A, symbol)
    print(f"El determinante es cero cuando {symbol} está en {critical_values}.")
    print("-" * 60)

    print("--- Análisis de los Casos Críticos (No hay solución única) ---")
    for val in critical_values:
        print(f"\nAnálisis para {symbol} = {val}:")

        A_sub = A_symbolic.subs(symbol, val)
        b_sub = b_symbolic.subs(symbol, val)
        augmented_matrix = A_sub.row_join(b_sub)
        print("Matriz aumentada para este valor:\n", augmented_matrix)

        rref_matrix, _ = augmented_matrix.rref()
        print("\nForma escalonada reducida por filas (RREF):\n", rref_matrix)

        inconsistent = any(all(elem == 0 for elem in row[:-1]) and row[-1] != 0 for row in rref_matrix.tolist())

        if inconsistent:
            print(f"\nConclusión (a): Para {symbol} = {val}, el sistema es inconsistente y NO TIENE SOLUCIONES.")
        else:
            print(f"\nConclusión (b): Para {symbol} = {val}, el sistema es consistente y tiene INFINITAS SOLUCIONES.")
    print("-" * 60)

    print("--- Análisis del Caso General (Solución única) ---")
    print(f"Existe una solución única siempre que {symbol} no sea uno de los valores críticos {critical_values}.")

    print("\nUsando la Regla de Cramer para encontrar la solución en términos del símbolo:")

    solutions = []
    for i in range(A_symbolic.cols):
        Ai = A_symbolic.copy()
        Ai[:, i] = b_symbolic

        det_Ai = Ai.det()
        xi = sympy.simplify(det_Ai / det_A)
        solutions.append(xi)
        print(f"  x{i+1} = det(A{i+1}) / det(A) = ({sympy.simplify(det_Ai)}) / ({sympy.simplify(det_A)}) = {xi}")

    print("\nConclusión (c): La solución única para un α dado (distinto de los valores críticos) es:")
    for i, sol in enumerate(solutions):
        print(f"  x{i+1} = {sol}")

alpha = sympy.Symbol('α')

A_system = sympy.Matrix([
    [1, -1, alpha],
    [-1, 2, -alpha],
    [alpha, 1, 1]
])

b_system = sympy.Matrix([-2, 3, 2])

analyze_system_with_parameter(A_symbolic=A_system, b_symbolic=b_system, symbol=alpha)
```

```

--- Análisis del Determinante ---
La matriz de coeficientes A es:
Matrix([[1, -1, α], [-1, 2, -α], [α, 1, 1]])

El determinante de A es: det(A) = 1 - α**2

Se busca para qué valores del símbolo el determinante es cero.
El determinante es cero cuando α está en [-1, 1].
-----
--- Análisis de los Casos Críticos (No hay solución única) ---

Análisis para α = -1:
Matriz aumentada para este valor:
Matrix([[1, -1, -1, -2], [-1, 2, 1, 3], [-1, 1, 1, 2]])

Forma escalonada reducida por filas (RREF):
Matrix([[1, 0, -1, -1], [0, 1, 0, 1], [0, 0, 0, 0]])

Conclusión (b): Para α = -1, el sistema es consistente y tiene INFINITAS SOLUCIONES.

Análisis para α = 1:
Matriz aumentada para este valor:
Matrix([[1, -1, 1, -2], [-1, 2, -1, 3], [1, 1, 1, 2]])

Forma escalonada reducida por filas (RREF):
Matrix([[1, 0, 1, 0], [0, 1, 0, 0], [0, 0, 0, 1]])

Conclusión (a): Para α = 1, el sistema es inconsistente y NO TIENE SOLUCIONES.
-----
--- Análisis del Caso General (Solución única) ---
Existe una solución única siempre que α no sea uno de los valores críticos [-1, 1].

Usando la Regla de Cramer para encontrar la solución en términos del símbolo:
x1 = det(A1) / det(A) = (-α - 1) / (1 - α**2) = 1/(α - 1)
x2 = det(A2) / det(A) = (1 - α**2) / (1 - α**2) = 1
x3 = det(A3) / det(A) = (α + 1) / (1 - α**2) = -1/(α - 1)

Conclusión (c): La solución única para un α dado (distinto de los valores críticos) es:
x1 = 1/(α - 1)
x2 = 1
x3 = -1/(α - 1)

```

Ejercicio 7

Repita el ejercicio 4 con el método Gauss-Jordan.

```

In [45]: import numpy as np

A = np.array([
    [1, 2, 0, 3],
    [1, 0, 2, 2],
    [0, 0, 1, 1]
])
x = np.array([1000, 500, 350, 400])
b = np.array([3500, 2700, 900])

consumo_actual = A @ x

print("Consumo actual de alimentos:", consumo_actual)
print("Suministro disponible:", b)

diferencia = b - consumo_actual
suficiente = np.all(diferencia >= 0)

if suficiente:
    print("\nRespuesta a: Sí hay suficiente alimento para el consumo actual.")
    print("Excedentes por tipo de alimento:", diferencia)
else:
    print("\nRespuesta a: No hay suficiente alimento para el consumo actual.")
    print("Déficits por tipo de alimento:", -diferencia[diferencia < 0])

```

```

Consumo actual de alimentos: [3200 2500 750]
Suministro disponible: [3500 2700 900]

```

```

Respuesta a: Sí hay suficiente alimento para el consumo actual.
Excedentes por tipo de alimento: [300 200 150]

```

```

In [47]: def max_incremento(A, b, x, especie):
    consumo_actual = A @ x
    excedente = b - consumo_actual

    a_j = A[:, especie]

    incrementos = excedente / a_j
    max_incremento = np.min(incrementos[a_j > 0])

```

```
return max_incremento
```

```
print("\nParte b: Máximo incremento individual por especie")
for j in range(len(x)):
    inc = max_incremento(A, b, x, j)
    print(f"Especie {j+1}: puede aumentar en {inc:.2f} unidades")
```

Parte b: Máximo incremento individual por especie

Especie 1: puede aumentar en 200.00 unidades

Especie 2: puede aumentar en 150.00 unidades

Especie 3: puede aumentar en 100.00 unidades

Especie 4: puede aumentar en 100.00 unidades

C:\Users\cuich\AppData\Local\Temp\ipykernel_9092\1507347974.py:7: RuntimeWarning: divide by zero encountered in divide
incrementos = excedente / a_j

In [48]: A_sin_1 = np.delete(A, 0, axis=1)
x_sin_1 = np.delete(x, 0)

```
consumo_sin_1 = A_sin_1 @ x_sin_1
excedente_sin_1 = b - consumo_sin_1
```

```
print("\nParte c: Extinción de especie 1")
print("Excedente disponible:", excedente_sin_1)
```

```
for j in range(len(x_sin_1)):
    inc = max_incremento(A_sin_1, b, x_sin_1, j)
    print(f"Especie {j+2}: puede aumentar en {inc:.2f} unidades")
```

Parte c: Extinción de especie 1

Excedente disponible: [1300 1200 150]

Especie 2: puede aumentar en 650.00 unidades

Especie 3: puede aumentar en 150.00 unidades

Especie 4: puede aumentar en 150.00 unidades

C:\Users\cuich\AppData\Local\Temp\ipykernel_9092\1507347974.py:7: RuntimeWarning: divide by zero encountered in divide
incrementos = excedente / a_j

In [49]: A_sin_2 = np.delete(A, 1, axis=1)
x_sin_2 = np.delete(x, 1)

```
consumo_sin_2 = A_sin_2 @ x_sin_2
excedente_sin_2 = b - consumo_sin_2
```

```
print("\nParte d: Extinción de especie 2")
print("Excedente disponible:", excedente_sin_2)
```

```
for j in range(len(x_sin_2)):
    especie_original = j if j < 1 else j + 1
    inc = max_incremento(A_sin_2, b, x_sin_2, j)
    print(f"Especie {especie_original+1}: puede aumentar en {inc:.2f} unidades")
```

Parte d: Extinción de especie 2

Excedente disponible: [1300 200 150]

Especie 1: puede aumentar en 200.00 unidades

Especie 3: puede aumentar en 100.00 unidades

Especie 4: puede aumentar en 100.00 unidades

C:\Users\cuich\AppData\Local\Temp\ipykernel_9092\1507347974.py:7: RuntimeWarning: divide by zero encountered in divide
incrementos = excedente / a_j